ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Relación de Problemas Tema 3 : Herencia (PDOO)

Índice

1. Ejercicio 1 **3**

2. Ejercicio 2 **3**

3. Ejercicio 3 **4**

4. Ejercicio 4 **6**

1 Ejercicio 1

Dada la siguiente clase en Java:

```java
public class Vertebrado
{
    public ArrayList<String> partesDelAbdomen();
    public void desplazarse();
    public String comunicarse(Vertebrado vertebrado);
    protected Vertebrado obtenerCopia();
}
```

Indica con una cruz en la casilla correspondiente según si la declaración de los siguientes métodos de la clase `Mamifero`, subclase de `Vertebrado`, sobrecargan (*overloading*) o redefinen (*overriding*) a los de la clase `Vertebrado`.

Método	**Sobrecarga**	**Redefinición**
public ArrayList<String>partesDelAbdomen()		X
public void desplazarse(Modo m)	X	
public String comunicarse(Vertebrado vertebrado)		X
public String comunicarse(Mamifero mamifero)	X	
public Vertebrado obtenerCopia()		X
protected Mamiero obtenerCopia()		X

Nota:Al cambiar únicamente la visibilidad o el tipo hace que sea una redefición (Últimos dos casos)

2 Ejercicio 2

A partir de las siguientes clases:

```java
public class Persona {
    protected String nombre;
    public Persona(String nom) { this.setNombre(nom); }
    protected String getNombre() { return this.nombre; }
    public String hablar() { return "bla bla bla"; }
}
```

```java
public class Estudiante extends Persona {
    public String carrera;
    public int curso;
    public Estudiante(String nom, String carr, int cur) {
        super(nom);
        carrera = carr;
        curso = cur;
    }
    public void estudiar() { System.out.println("estudiando"); }
}
```

Implementa la clase `EstudianteInformatica` que hereda de `Estudiante`. Debe tener:

- Una nueva variable de instancia que es una colección de `String` con los dispositivos que utiliza (por ejemplo, `PC`, `tablet`, `smartphone`).
- Métodos para consultar esa variable.
- Una redefinición del método `estudiar()` para que además de estudiar como los otros alumnos, diga que estudia con el último dispositivo de su colección.
- Un constructor para inicializar la clase.

Implementa este problema en Java y Ruby:

```java
    public class EstudianteInformatica extends Estudiante {
        private ArrayList<String> dispositivos;
        public EstudianteInformatica(string nombre, string curso){
            super(nombre, "Informatica", curso);
            dispositivos = new ArrayList<String>();
        }

        @Override
        public void Estudiar(){
            System.out.println("Estudiando Informatica con " + dispositivos
                .get(dispositivos.size() - 1));
        }
    } // Fin de la clase EstudianteInformatica
```

Listing 1: Clase Estudiante de Informática

```ruby
    def EstudianteInformatica < Estudiante
        def initialize (nombre, curso)
            super(nombre, "Informatica", curso)
            @dispositivos = [] # @dispositivos = Array.new
        end

        def Estudiar
            super.Estudiar
            puts "Estudiando Informatica con #{@dispositivos[-1]}"
        end
    end
```

Listing 2: Clase Estudiante de Informática

3 Ejercicio 3

Nota: En las diapositivas de la Relación hay un error, donde pone 3, no es el ejercicio 3 es la parte 2 del ejercicio 2, es decir, otra clase, el ejercicio 3 es el marcado con el número 4.

Dada la siguiente clase abstracta en Java:

```java
abstract class Transporte {
    protected String marca;

    protected String getMarca() {
        return this.marca;
```

```java
    }

    protected void setMarca(String marc) {
        this.marca = marc;
    }

    public abstract String hacerRuta(String origen, String destino);
}
```

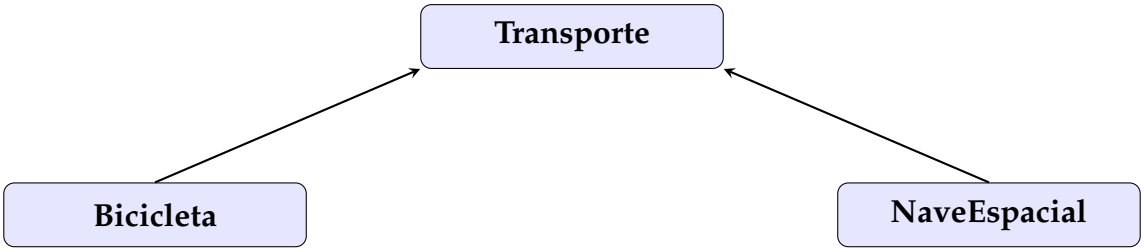
Crea dos nuevas clases, `Bicicleta` y `NaveEspacial`, que hereden de `Transporte` e implementen el método `hacerRuta` de forma diferente:

```java
    public class Bicicleta extends Transporte {
        @Override
        public String hacerRuta(String origen, String destino) {
            return "Pedaleando desde " + origen + " hasta " + destino;
        }
    }

    def NaveEspacial < Transporte
        def hacerRuta(origen, destino)
            "Volando desde #{origen} hasta #{destino} por el espacio"
        end
    end
```

Listing 3: Clase Bicicleta

Diagrama UML



4 Ejercicio 4

Hacer el ejercicio anterior pero en Ruby.

```ruby
def Bicicleta < Transporte
    def hacerRuta(origen, destino)
        "Pedaleando desde #{origen} hasta #{destino}"
    end
end

def NaveEspacial < Transporte
    def hacerRuta(origen, destino)
        "Volando desde #{origen} hasta #{destino} por el espacio"
    end
end
```

Listing 4: Clase Bicicleta